

Encryption Standards

Owner: Mark Stolte, Founder — markstolte02@gmail.com

Last reviewed: 2026-07-05

Stage: Pre-launch / active development

Purpose

States how Penny Pilot encrypts data in transit, at rest, and on-device, with specific reference to the current implementation rather than generic requirements.

In transit

- All communication with Supabase (Postgres via PostgREST, Auth, Edge Functions) and with Plaid's API happens over HTTPS/TLS, as provided by those vendors' SDKs and endpoints. The app does not make cleartext HTTP calls to any backend.
- The iOS app declares `ITSAppUsesNonExemptEncryption: false` (`app.json`), meaning it does not implement its own proprietary cryptography beyond the standard TLS/HTTPS handled by the OS and by Apple/Google/Supabase/Plaid SDKs — this keeps the app out of U.S. export compliance documentation requirements that apply to custom encryption.
- Plaid Link's OAuth redirect uses an HTTPS Universal Link (`https://penny-pilot.net/plaid/oauth`), not a custom URL scheme, per Plaid's requirement that OAuth redirects be HTTPS (`docs/plaid-sandbox-setup.md`).

At rest

- **Plaid access tokens** are the most sensitive value Penny Pilot stores. Before being written to `plaid_items.access_token_ciphertext`, they are encrypted with **AES-256-GCM** using a 256-bit key (`PLAID_TOKEN_ENCRYPTION_KEY`, generated with `openssl rand -base64 32`) that exists only as a Supabase Edge Function secret (`supabase/functions/_shared/plaid.ts`). A random 12-byte IV is generated per encryption and stored alongside the ciphertext (`iv.ciphertext`, both base64). Decryption happens only inside the `plaid-sync-transactions` Edge Function, server-side; the raw Plaid access token is never sent to or stored by the mobile client.
- **Everything else in Postgres** (transactions, accounts, categories, goals, profile data) relies on Supabase's underlying storage-level encryption at rest, which is standard for the hosting platform. Row Level Security (see [Access Control Policy](#)) is the primary control limiting *who* can query that data; disk-level encryption protects against storage-media compromise.
- Supabase Edge Function secrets (`PLAID_CLIENT_ID`, `PLAID_SECRET`, `PLAID_TOKEN_ENCRYPTION_KEY`, etc.) are stored via `supabase secrets set` and are not present in the Postgres database, in the mobile bundle, or in git.

On-device

- Supabase session tokens are stored via `expo-secure-store`, which uses the iOS Keychain and Android Keystore (`src/services/supabase-client.ts`). On web, the same adapter falls back to `localStorage` — accepted only as a local-development convenience, not the production auth path, since the shipped product is the native app.
- In `mock` mode, or for locally-imported CSV/XLS transactions before a user links Supabase, financial data is held in `AsyncStorage` on-device. `AsyncStorage` is not encrypted by default on either platform; this is an accepted limitation for the local-first/no-account path today (the same tier of data a user's own banking app or spreadsheet would hold locally) and is revisited if that mode is retained as a permanent, no-signup product option rather than a development convenience.

Key management

- The Plaid token-encryption key is generated once via `openssl rand -base64 32` and set as a Supabase secret (`docs/plaid-sandbox-setup.md`). It is not rotated automatically today; rotation procedure (re-encrypt all `plaid_items` rows with a new key, or force re-linking) is defined in the [Incident Response Plan](#) for the case where rotation is required after a suspected compromise.
- No cryptographic key is ever committed to source control. `.env` is git-ignored; `.env.example` contains only placeholder values.

What's explicitly out of scope

Penny Pilot does not implement its own TLS termination, certificate management, or HSM-backed key storage — those are inherited from Supabase's and Plaid's infrastructure. This is a deliberate build-vs-buy choice appropriate to team size, not an oversight.